



SMART QR-BASED RESTAURANT ORDERING & KITCHEN MANAGEMENT SYSTEM

Team - 25 || Section - 12 || CSIT

Hitesh Suthar - 2520090078 || V. Nikith Chowdary - 2520090219 || B. Vasudeva - 2520090136

Guide - Dr. Khajashareef

Project Domain: Full-Stack Web Application | Platform: Responsive Web

Modules: Customer Ordering | Chef Kitchen Dashboard | Manager Dashboard

Abstract

- Web-based restaurant ordering system using table-specific QR codes.
- Customers browse the live menu, build a cart, place orders and track status.
- Chefs receive live orders; managers control menu, tables, QR codes and KPIs.
- Outcome: faster ordering, fewer manual steps and centralized restaurant operations.

Problem Statement & Objectives

- Problem: Manual order taking can be slow, error-prone and dependent on staff availability.
- Objective 1: Enable QR-based table validation and digital menu access.
- Objective 2: Securely create orders with server-side price and 5% tax calculation.
- Objective 3: Provide real-time kitchen and customer order status updates.
- Objective 4: Centralize menu, category, table, order and KPI management.
- Scope: Customer, Chef and Manager workflows with authenticated staff access.

Existing vs Proposed System

Feature	Existing	Proposed
Ordering	Manual / staff	QR self-order
Menu	Printed / static	Live digital
Kitchen	Paper / verbal	Socket.IO live queue
Tracking	Ask staff	Real-time status
Management	Fragmented	Central dashboard

Feasibility Study

- Technical: React, Node.js, Express, MongoDB and Socket.IO are established web technologies.
- Economic: Uses widely available, open-source development tools and libraries.
- Operational: Separate Customer, Chef and Manager workflows simplify daily use.
- Schedule: Modular frontend/backend structure supports iterative development and testing.

Validation & Testing

- QR validation rejects invalid or inactive tables.
- Server validates menu availability, quantity and order totals.
- JWT-protected Manager/Chef routes enforce roles.
- Socket.IO supports order-new and statusUpdate events.
- Order state machine blocks backward status changes.

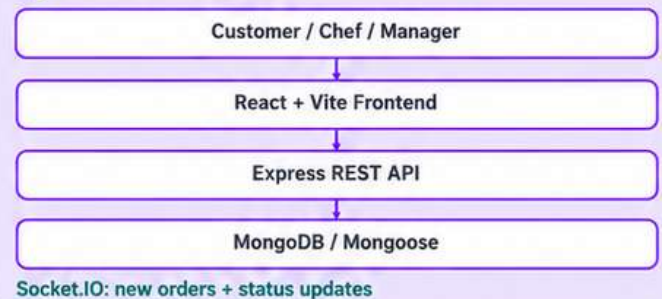
Conclusion

- Connects customer ordering, kitchen operations and management in one platform.
- Reduces manual coordination and improves order visibility.
- Combines real-time communication with secure server-side processing.

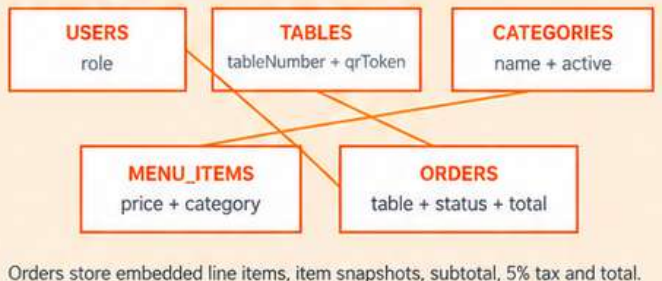
Tools & Technologies

Frontend: React 19, Vite, React Router, Axios, Lucide React
Backend: Node.js, Express.js, REST APIs
Database: MongoDB, Mongoose
Real-Time: Socket.IO for kitchen and order updates
Security: JWT + HTTP-only cookies, Argon2, Helmet, CORS, rate limiting
QR: qrcode.react for table QR generation

System Architecture



ER Diagram & Database Design



Implementation Screenshots

